



程序设计原理

循环结构

任课教师：喻 梅
于瑞国
王建荣
李雪威
赵满坤



Three Basic Control Structures of Programs

- All programs can be composed from these three structures.

1) Sequential · 顺序结构

Statements execute one after another in source order.

```
// Example
int a,b; cin >> a >> b;
int s = a + b;
cout << s << endl;
```

2) Selection / Branching · 分支结构

Choose a path based on condition(s) using `if/else` or `switch`.

```
// Example
int score; cin >> score;
if (score >= 60) cout << "Pass";
else cout << "Fail";
```

3) Looping / Iteration · 循环结构

Repeat a block while a condition holds (or for a known count).

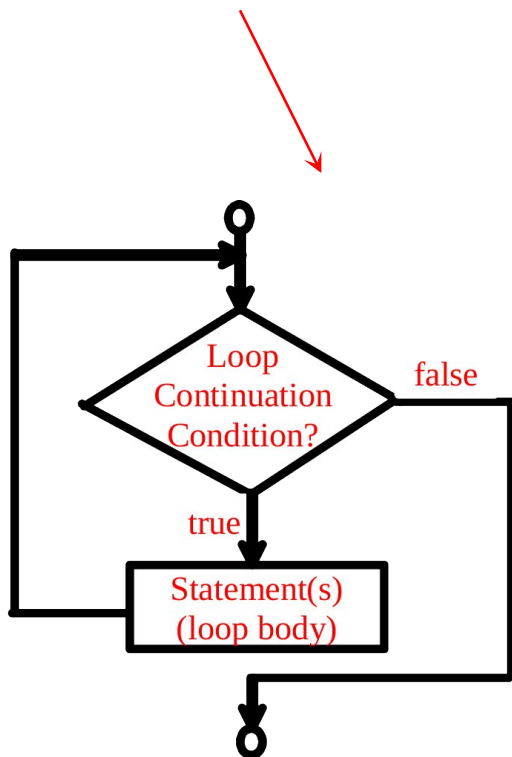
```
// Example
int n; cin >> n;
for (int i=1;i<=n;i++) cout << i << " ";
```

All complex programs are built by combining these three structures. Next: focus on **Selection (分支)**.



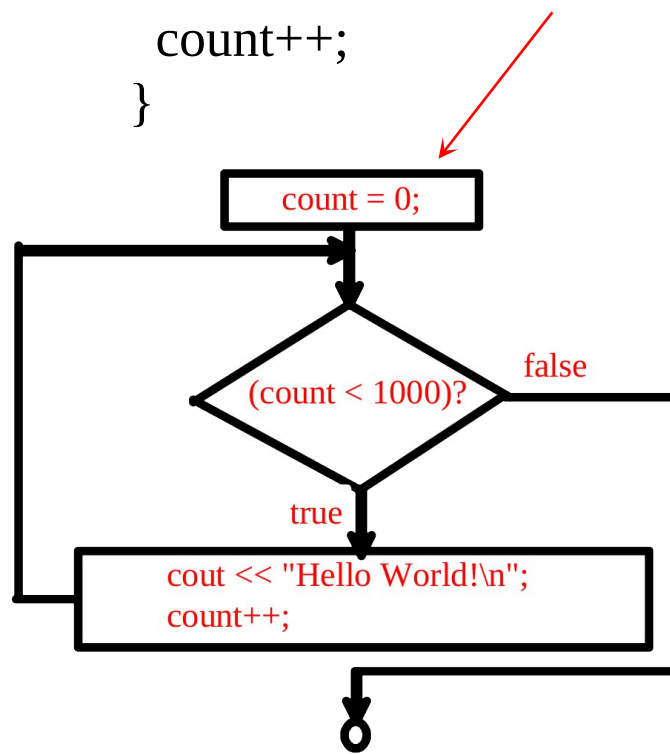
while Loop Flow Chart

```
while (loop-continuation-condition)
{
    // loop-body;
    Statement(s);
}
```



(a)

```
int count = 0;
while (count < 1000)
{
    cout << "Hello World!\n";
    count++;
}
```



(b)



The While Loop

```
int sum = 0, i = 1;
while(i<10)
{
    sum = sum + i;
    i++;
}
cout << "sum is " << sum;
```

```
int sum = 0, i = 1;
while(i<10)
{
    sum = sum + i;
}
cout << "sum is " << sum;
```



infinite loop



The While Loop

```
int count = 0;
while (count<=100)
{
    cout<<"Welcom to C++!\n";
    count++;
}
```

The loop execute 101 times.

Don't use floating-point values for equality checking in a loop control. Since floating-point values are approximations, using them could result in imprecise counter values and inaccurate results. This example uses int value for data. If a floating-point type value is used for data, (data != 0) may be true even though data is 0.

[illegible]



Example

- 输入一个整数n，计算1到n的数字之和。

```
#include <iostream>
using namespace std;

int main() {

    int n, sum = 0, i = 1;

    cin >> n;

    // 使用while循环计算1到n的和
    while (i <= n) {
        sum += i; // 将i累加到sum
        i++;      // i增加1
    }

    cout << "1到" << n << "的和是： " << sum << endl;

    return 0;
}
```



Using break and continue

```
#include <iostream>
using namespace std;
int main()
{
    int sum = 0;
    int number = 0;

    while (number < 20) {
        number++;
        sum += number;
        if (sum >= 100) break;
    }
    cout << "The number is " << number << endl;
    cout << "The sum is " << sum << endl;

    return 0;
}
```




Using break and continue

```
#include <iostream>
using namespace std;

int main()
{
    int sum = 0;
    int number = 0;

    while (number < 20) {
        number++;
        if (number == 10 || number == 11) continue;
        sum += number;
    }

    cout << "The sum is " << sum;
    return 0;
}
```



Exercise

Print Test Cases

In the online judge system, a judge file may include multiple datasets to check whether the submitted program outputs a correct answer for each test case. This task is to practice solving a problem with multiple datasets.

Write a program which reads an integer x and print it as is. Note that multiple datasets are given for this problem.

Input

The input consists of multiple datasets. Each dataset consists of an integer x in a line.

The input ends with an integer 0. Your program should not process (print) for this terminal symbol.



Exercise

Output

For each dataset, print x in the following format:

Case i: x

where i is the case number which starts with 1. Put a single space between "Case" and i. Also, put a single space between ':' and x.

Constraints

$1 \leq x \leq 10000$

The number of datasets ≤ 10000

Sample Input

3

5

11

7

8

19

0

Sample Output

Case 1: 3

Case 2: 5

Case 3: 11

Case 4: 7

Case 5: 8

Case 6: 19



Exercise

Swapping Two Numbers

Write a program which reads two integers x and y , and prints them in ascending order.

Input

The input consists of multiple datasets.

Each dataset consists of two integers x and y separated by a single space.

The input ends with two 0 (when both x and y are zero). Your program should not process for these terminal symbols.

Output

For each dataset, print x and y in ascending order in a line. Put a single space between x and y .

Constraints

$0 \leq x, y \leq 10000$, the number of datasets ≤ 3000

Sample Input

3 2

2 2

5 3

0 0

Sample Output

2 3

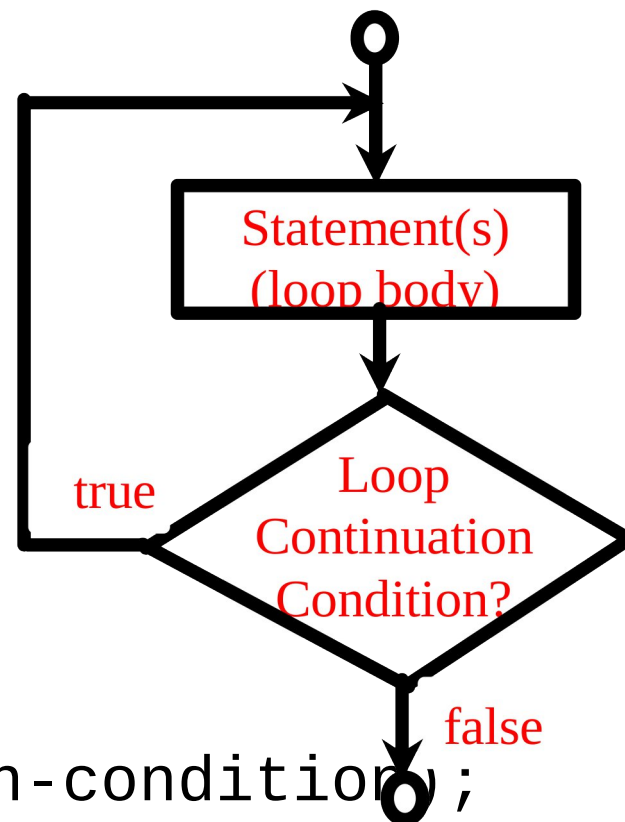
2 2

3 5



do-while Loop

```
do  
{  
    // Loop body;  
    Statement(s);  
} while (loop-continuation-condition);
```





Example

➤ While Solution

```
int sum=0;
int counter=1;
while(counter<=n)
{
    sum=sum+counter;
    counter++;
}
```

➤ Do-While Solution

```
int sum=0;
int counter=1;
do
{
    sum=sum+counter;
    counter++;
} while(counter<=n);
```



while Loop

```
int count ;
```

```
count = 4;
```

```
while (count > 0);
```

```
{
```

```
    cout << count << endl ;
```

```
    count - - ;
```

```
}
```

```
cout << "Done" << endl ;
```



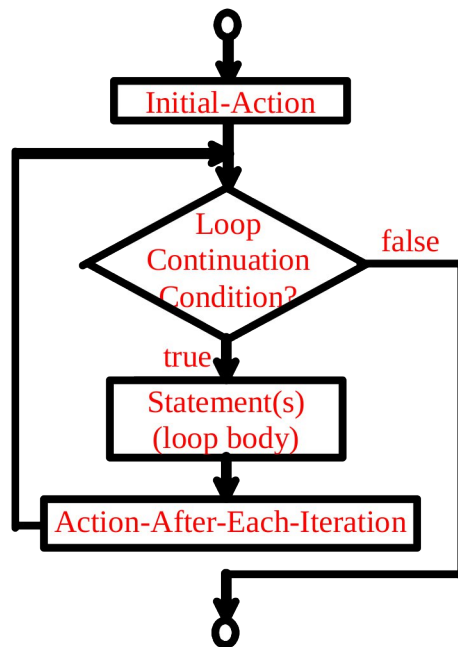
infinite loop



for Loops

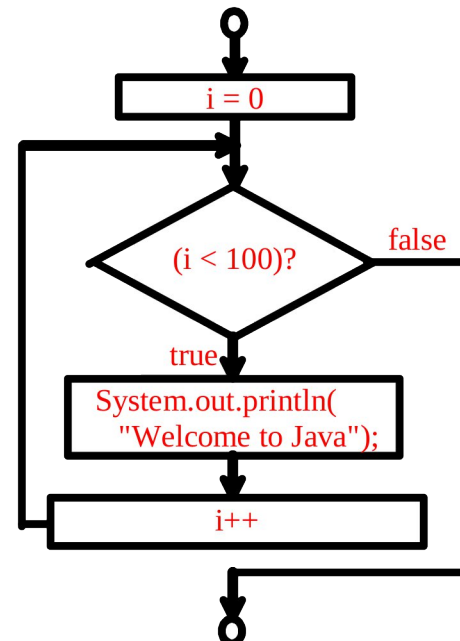
for (initial-action; loop-continuation-condition; action-after-each-iteration)

```
{  
  // loop body;  
  Statement(s);  
}
```



(A)

```
int i;  
for (i = 0; i < 100; i++)  
{  
  cout << "Welcome to C++!\n";  
}
```



(B)



Note

The initial-action in a for loop can be a list of zero or more comma-separated expressions. The action-after-each-iteration in a for loop can be a list of zero or more comma-separated statements. Therefore, the following two for loops are correct. They are rarely used in practice, however.

```
for (int i = 1; i < 100; cout << (i++));
```

```
for (int i = 0, j = 0; (i + j < 10); i++, j++) {  
    // Do something  
}
```



Example

- 输入一个整数n，计算1到n的数字之和。

```
#include <iostream>
using namespace std;
```

```
int main() {
```

```
    int n, sum = 0;
```

```
    cin >> n;
```

```
    // 使用for循环，计算1到n的和
```

```
    for (int i = 1; i <= n; i++) {
```

```
        sum += i;
```

```
    }
```

```
    cout << "1到" << n << "的和是： " << sum << endl;
```

```
    return 0;
```

```
}
```



Exercise

How Many Divisors?

Write a program which reads three integers a , b and c , and prints the number of divisors of c between a and b .

Input

Three integers a , b and c are given in a line separated by a single space.

Output

Print the number of divisors in a line.

Constraints

$1 \leq a, b, c \leq 10000$

$a \leq b$

Sample Input 1

5 14 80

Sample Output 1

3



Exercise

Simple Calculator

Write a program which reads two integers a , b and an operator op , and then prints the value of $a \text{ op } b$.

The operator op is '+', '-', '*' or '/' (sum, difference, product or quotient). The division should truncate any fractional part.

Input

The input consists of multiple datasets. Each dataset is given in the following format.

$a \text{ op } b$

The input ends with a dataset where $op = '?'$.

Your program should not process for this dataset.

Output

For each dataset, print the value in a line.

Constraints

$$0 \leq a, b \leq 20000$$

No divisions by zero are given.

Sample Input 1

1 + 2

56 - 18

13 * 2

100 / 10

27 + 81

0 ? 0

Sample Output 1

3

38

26

10

108



Exercise

Min, Max and Sum

Write a program which reads a sequence of n integers $a_i (i=1,2,\dots,n)$, and prints the minimum value, maximum value and sum of the sequence.

Input

In the first line, an integer n is given. In the next line, n integers a_i are given in a line.

Output

Print the minimum value, maximum value and sum in a line. Put a single space between the values.

Constraints

$$0 < n \leq 10000$$

$$-1000000 \leq a_i \leq 1000000$$

Sample Input

5

10 1 5 4 17

Sample Output

1 17 37